NAME: VIJAYALAKSHMI S

REG NO:192472326

Q1) Write program demonstrates how to use regular expressions in Python to match and search for patterns in text.

Programiz
Python Online Compiler

main.py

Visualize

Run

```
import re

text = "Python programming is easy"

a = re.match("Python", text)
b = re.search("easy", text)

if a:
    print("Match found:", a.group())
else:
    print("No match found")

if b:
    print("Search found:", b.group())
else:
    print("No search found")
```

Output

```
Match found: Python
Search found: easy
```

Q2) Implement a basic finite state automaton that recognizes a specific language or pattern. In this example, we'll create a simple automaton to match strings ending with 'ab' using python.

main.py

Visualize

Run

```python
def fsa(s):
    state = 0

    for ch in s:
        if ch == 'a':
            state = 1
        elif ch == 'b':
            if state == 1:
                state = 2
            else:
                state = 0
        else:
            return False

    return state == 2

s = input("Enter a string containing a and b: ")

if fsa(s):
    print("String Accepted")
else:
    print("String Rejected")
```

Output

```
Enter a string containing a and b: abab
String Accepted
```

Q3) Write program demonstrates how to perform morphological analysis using the NLTK library in Python.

main.py

Visualize

Run

```
word = input("Enter a word: ")

if word.endswith("ies"):
    base = word[:-3] + "y"
elif word.endswith("ing"):
    base = word[:-3]
elif word.endswith("ed"):
    base = word[:-2]
elif word.endswith("s") and not word.endswith("ss"):
    base = word[:-1]
else:
    base = word

print("Base form:", base)
```

Output

```
Enter a word: cities
Base form: city
```

Q4) Implement a finite-state machine for morphological parsing. In this example, we'll create a simple machine to generate plural forms of English nouns using python.

main.py

Visualize

Run

```python
def plural(word):
    if word.endswith(("s", "sh", "ch", "x", "z")):
        return word + "es"
    elif word.endswith("y") and len(word) > 1 and word[-2] not in "aeiou":
        return word[:-1] + "ies"
    else:
        return word + "s"

word = input("Enter a singular noun: ")
print("Plural form:", plural(word))
```

Output

```
Enter a singular noun: city
Plural form: cities
```

Q5) Use the Porter Stemmer algorithm to perform word stemming on a list of words using python libraries.

main.py

Visualize

Run

```python
def stem(word):
    if word.endswith("ies"):
        return word[:-3] + "i"
    elif word.endswith("ing"):
        return word[:-3]
    elif word.endswith("ed"):
        return word[:-2]
    elif word.endswith("s"):
        return word[:-1]
    else:
        return word

words = ["playing", "played", "plays", "studies", "running"]

for word in words:
    print(word, "->", stem(word))
```

Output

```
playing -> play
played -> play
plays -> play
studies -> studi
running -> runn
```

Q6) Implement a Basic N-gram Model for Text Generation (Bigram)

main.py

Visualize

Run

```
import random

text = "the cat is cute the cat is small the dog is cute"
words = text.split()

bigrams = {}
for i in range(len(words) - 1):
    bigrams.setdefault(words[i], []).append(words[i + 1])

word = "the"
result = [word]

for i in range(8):
    if word not in bigrams:
        break
    word = random.choice(bigrams[word])
    result.append(word)

print("Generated Text:", " ".join(result))
```

Output

```
Generated Text: the cat is cute the dog is cute the
```

Q7) Part-of-Speech Tagging Using NLTK

main.py

Visualize

Run

```
import re

text = "The smart student reads a book"
words = re.findall(r"\b\w+\b", text)

def pos_tag(word):
    w = word.lower()

    if w in ["the", "a", "an"]:
        return "DT"
    elif w in ["is", "am", "are", "read", "reads", "run", "runs"]:
        return "VB"
    elif w.endswith("ly"):
        return "RB"
    elif w in ["smart", "beautiful", "good", "happy"]:
        return "JJ"
    else:
        return "NN"

for word in words:
    print(word, ":", pos_tag(word))
```

Output

```
The : DT
smart : JJ
student : NN
reads : VB
a : DT
book : NN
```

Q8) Simple Stochastic POS Tagging Using Probabilities

main.py

Visualize

Run

```
import random

words = ["dogs", "run", "beautiful"]
prob = {
    "dogs": {"Noun": 0.9, "Verb": 0.1},
    "run": {"Verb": 0.7, "Noun": 0.3},
    "beautiful": {"Adjective": 0.95, "Noun": 0.05}
}

for word in words:
    tags = list(prob[word].keys())
    weights = list(prob[word].values())
    tag = random.choices(tags, weights=weights)[0]
    print(word, ":", tag)
```

Output

```
dogs : Noun
run : Verb
beautiful : Adjective
```

Q9) Rule-Based POS Tagging Using Regular Expressions

main.py

Visualize

Run

```
import re

def pos_tag(word):
    if re.match(r".*ing$", word):
        return "Verb"
    elif re.match(r".*ly$", word):
        return "Adverb"
    elif re.match(r".*ous$|.*ful$", word):
        return "Adjective"
    elif re.match(r".*s$", word):
        return "Noun (Plural)"
    else:
        return "Noun"

words = ["running", "quickly", "beautiful",
         "cats", "book"]

for word in words:
    print(word, ":", pos_tag(word))
```

Output

```
running : Verb
quickly : Adverb
beautiful : Adjective
cats : Noun (Plural)
book : Noun
```

Q10) Transformation-Based POS Tagging

main.py

Visualize

Run

```python

words = ["playing", "quickly", "cats", "happy"]
tags = ["Noun", "Noun", "Noun", "Noun"]

for i, word in enumerate(words):
    if word.endswith("ing"):
        tags[i] = "Verb"
    elif word.endswith("ly"):
        tags[i] = "Adverb"
    elif word.endswith("s"):
        tags[i] = "Plural Noun"
    elif word.endswith("y"):
        tags[i] = "Adjective"

for word, tag in zip(words, tags):
    print(word, ":", tag)

```

Output

```
playing : Verb
quickly : Adverb
cats : Plural Noun
happy : Adjective
```